

AI Development & Decision Log

This log captures key prompts, architectural decisions, what was questioned, iterations, and deliberate engineering judgments throughout the development of the **Document Intake Assistant**.

Log Entry 001 — Project Inception & Implementation Strategy

Prompt / Task

Review the Wenup Technical Test Assignment, Revised PRD v2, and Feature-by-Feature Implementation & Testing Guide v2. Establish repository rules, strict feature-by-feature execution protocol (`IMPLEMENT` → `MANUAL CHECK` → `WRITE TESTS` → `ALL PASS` → `COMMIT` → `NEXT FEATURE`), and ensure no scope creep or framework bloat.

Output that was useful

- Feature breakdown from F01 to F26.
- Architecture defining LLM for language extraction vs deterministic domain layer for state, planning, and document rendering.
- Defined 8 golden failure/edge scenarios.

What I questioned

- *Should we use LangGraph or an agent framework?*
Answer: No. PRD explicitly rejects LangGraph for this assignment. Explicit Python orchestration, Pydantic schemas, and a deterministic reducer are clearer, more testable, and less prone to uncontrolled loops.
- *Should the LLM return the whole new state or state operations?*
Answer: LLM returns operations with grounded evidence (`op` , `field` , `value` , `evidence`). Domain reducer validates and mutates state. This prevents model hallucinations from corrupting application state.
- *Should document preview be LLM-generated?*
Answer: No. Document generation must be 100% deterministic based only on confirmed structured state, with explicit legal disclaimers.

Decision

Follow the exact build order:

- **Phase 1: Core Engine** (F01–F14)
- **Phase 2: API, Sessions & Concurrency** (F15–F17)
- **Phase 3: Functional Frontend** (F18–F22)
- **Phase 4: Testing, Evaluation & Submission** (F23–F26)

Proceed feature-by-feature, validating with tests and commits at each step.

Log Entry 002 — F01 Project Foundation

Prompt / Task

Implement F01: FastAPI application skeleton, Pydantic settings configuration (`app/config.py`), `.env.example` , dependencies (`requirements.txt`), and health endpoint (`GET /api/health`).

Output that was useful

- Clean FastAPI application structure with environment variable management via `pydantic-settings` .
- Health endpoint reporting uptime status, configuration mode (`mock` by default), and timestamp.

What I questioned

- *Should we include heavy optional dependencies like `uvloop` in `requirements.txt` ?*
Answer: No. `uvloop` slowed down installation and added unnecessary native build complexity. Standard Python `asyncio` + `uvicorn` is fast, portable, and cleanly satisfies the requirements.

Decision

Use standard `uvicorn` and pure-Python async client testing with `httpx` + `pytest-asyncio` .

Result

- Application boots cleanly.
- `GET /api/health` , `/` , and `/docs` all return HTTP 200.
- All 3 tests in `tests/test_health.py` pass.

Log Entry 003 — F02 Domain State Model

Prompt / Task

Implement F02: Structured source of truth in `app/domain/models.py` using explicit `FieldStatus` (`UNKNOWN` , `UNCONFIRMED` , `CONFIRMED` , `NOT_APPLICABLE`) and strongly typed models for all assignment fields, changes, clarifications, and sessions.

Output that was useful

- Explicit `Field[T]` generic model capturing value, status, evidence, and turn.
- Models for `Executor` , `Gift` , `WishesState` , `Change` , `PendingClarification` , `Message` , and `Session` .

What I questioned

- *Should conversation history hold the current state values?*
Answer: No. PRD guardrail states conversation history is strictly evidence/audit context. `WishesState` is the sole source of truth.

Decision

Model all fields explicitly in `wishesState` and track mutations via immutable `Change` audit logs.

Result

- Domain models created with clear status transitions.
 - All 7 new unit tests in `tests/unit/test_domain_models.py` passed (10/10 total tests pass).
-

Log Entry 004 — F03 State Reducer + Validation

Prompt / Task

Implement F03: Single deterministic state reducer in `app/domain/reducer.py` with validation rules in `app/domain/validators.py`. Support `set`, `add`, `remove`, `clear`, `confirm` across all whitelisted fields with strict type checking and change auditing.

Output that was useful

- Whitelist-driven validation (`ALLOWED_FIELDS`) rejecting unauthorized paths and illegal operations before state changes occur.
- Immutable state copies (`model_copy(deep=True)`) preventing unintended mutations.
- Batch atomic execution (`apply_operations`).

What I questioned

- *How should item deletion from complex lists like `specific_gifts` work when a user only specifies the item name?*

Answer: Allow `remove` operation on `specific_gifts` to accept either full `Gift` dicts or string item names (e.g. `"Vintage Guitar"`).

Decision

Support string identifiers for `remove` operations in list fields and ensure idempotent `set` operations do not create redundant change records.

Result

- 9 unit tests created in `tests/unit/test_reducer.py` verifying field acceptance, unknown field rejection, type mismatches, and immutability.
 - All 19 tests in the test suite pass.
-

Log Entry 005 — F04 Explicit Correction Semantics

Prompt / Task

Implement F04: Handle explicit user corrections (e.g., "Actually, Bob is my executor, not James"), replace confirmed values directly without getting stuck in clarification loops, record audit events with `kind="correction"`, and clear active pending clarifications.

Output that was useful

- Clean separation between normal `set` operations and `correction` mutations.
- `apply_operations_to_session` helper that clears pending clarification once the referenced field is updated or corrected.

What I questioned

- *Should an explicit correction require confirmation before being accepted?*

Answer: No. An explicit correction from the user is direct intent. Requiring confirmation would create an unnecessary clarification loop. It should immediately replace the value and record the old vs new values in the change log.

Decision

Apply corrections immediately, record `kind="correction"` with `old_value` and `new_value`, and clear pending clarifications for that field.

Result

- 3 new unit tests in `tests/unit/test_corrections.py` covering scalar corrections, nested structure corrections, and session-level clarification resolution.
- All 22 tests pass.

Log Entry 006 — F05 Contradiction Handling

Prompt / Task

Implement F05: Contradiction detection engine in `app/domain/contradictions.py`. Ensure unacknowledged conflicting statements against confirmed state (e.g. `has_children=False` vs mentioning a son/daughter) do not silently overwrite confirmed facts, but instead generate a `PendingClarification` and pause field progression.

Output that was useful

- `detect_contradictions` inspecting proposed operations and user text against confirmed state.
- Distinct handling where `is_correction=True` allows direct updates while `is_correction=False` on conflicting fields triggers clarification.

What I questioned

- Should a semantic mention like 'leave my car to my son' be caught if the LLM didn't emit a `children` operation?

Answer: Yes. Cross-referencing child-related keywords when `has_children` is confirmed `False` prevents silent inconsistency in the final document.

Decision

Enforce strict contradiction protection: keep confirmed state unchanged, create `PendingClarification`, and require explicit user resolution before changing the underlying fact.

Result

- 4 unit tests created in `tests/unit/test_contradictions.py`.
- All 26 tests in the test suite pass.

Log Entry 007 — F06 Conditional State Rules

Prompt / Task

Implement F06: Conditional state rules and cascades in `app/domain/rules.py` (e.g., `has_children=False` cascades `children` to `NOT_APPLICABLE`, while `has_children=True` resets `children` to `UNKNOWN` if missing; explicitly confirmed empty lists for "no gifts" vs `UNKNOWN`).

Output that was useful

- Pure function `apply_conditional_rules(state: WishesState)` seamlessly incorporated into the reducer pipeline.
- Distinction between `UNKNOWN` (must be prompted) and `CONFIRMED` empty list `[]` (user deliberately stated "no gifts").

What I questioned

- If the user corrects `has_children` from `False` back to `True`, what happens to `children`?

Answer: `children` must automatically transition from `NOT_APPLICABLE` back to `UNKNOWN` so the planner can ask for the children's names.

Decision

Enforce deterministic rule cascades post-operation so invalid state combinations (e.g. `has_children=False` with `children` remaining `UNKNOWN`) are impossible.

Result

- 5 unit tests in `tests/unit/test_rules.py` covering all cascades and empty list transitions.
- All 31 tests pass.

Log Entry 008 — F07 Deterministic Planner

Prompt / Task

Implement F07: Deterministic Planner in `app/domain/planner.py` to decide **WHAT** action happens next (resolving contradictions, confirming unconfirmed fields, asking for missing fields in strict order, or completing the document).

Output that was useful

- `PlannerAction` enum representing all valid workflow actions.
- Priority engine: `RESOLVE_CONTRADICTION` $\rightarrow$ `CONFIRM_UNCONFIRMED` $\rightarrow$ Next missing field $\rightarrow$ `COMPLETE`.
- Helpers `get_missing_fields` and `is_state_complete`.

What I questioned

- *Should the LLM decide which field to ask next?*
Answer: No. Deterministic planner owns workflow decisions. This guarantees zero skipped required fields, no hallucinated steps, and predictable interview flow.

Decision

Keep planner logic 100% deterministic outside the LLM. The LLM will only be responsible for phrasing (F13).

Result

- 7 unit tests created in `tests/unit/test_planner.py`.
- All 38 tests in the test suite pass.

Log Entry 009 — F08 LLM Interface + Mock Provider

Prompt / Task

Implement F08: LLM provider abstraction in `app/llm/base.py`, deterministic offline `MockLLMClient` in `app/llm/mock.py`, and `GeminiClient` in `app/llm/gemini.py`. Ensure the entire suite can run without live API keys and errors map to typed application exceptions.

Output that was useful

- `LLMClient` Protocol defining `extract` and `compose`.
- `MockLLMClient` with pattern-based extraction for all intake fields, scriptable responses, and error simulations.
- `GeminiClient` with timeout handling and error mapping (`LLMNotConfiguredError`, `LLMProviderError`, `LLMTimeoutError`).

What I questioned

- *Should tests require network access or a live Gemini API key?*

Answer: No. The PRD explicitly requires deterministic offline testing via `MockLLMClient`. Live keys are purely optional for real-world execution.

Decision

Use `MockLLMClient` as the default test and development client so the application works out-of-the-box without setup barriers.

Result

- 6 unit tests in `tests/unit/test_llm_clients.py` verifying mock extraction, composing, scripted errors, and unconfigured Gemini key behavior.
 - All 44 tests pass.
-

Log Entry 010 — F09 Structured LLM Extraction

Prompt / Task

Implement F09: Strict structured LLM extraction schema (`ExtractionResult`, `ExtractionOperation`, `ExtractionAmbiguity`, `ExtractionContradiction`) in `app/llm/schemas.py` and structured extraction prompts in `app/llm/prompts.py` with prompt-injection defense boundaries.

Output that was useful

- Pydantic models validating `op`, `field`, `value`, `evidence`, `confidence`, and `is_correction` before operations reach the domain reducer.
- Security instructions in prompts ensuring user messages are treated strictly as passive data.

What I questioned

- *What if the LLM produces extra fields or invalid operation names?*

Answer: Strict Pydantic validation rejects the schema and triggers the recovery flow (F11) rather than letting invalid data pollute state.

Decision

Treat LLM output as untrusted external input and enforce strict schema parsing with evidence grounding requirements.

Result

- 4 unit tests in `tests/unit/test_extraction_schemas.py` verifying valid schemas and rejection of malformed operations.
 - All 48 tests pass.
-

Log Entry 011 — F10 Evidence Grounding Validation

Prompt / Task

Implement F10: Robust evidence grounding validator in `app/llm/grounding.py`. Verify that all extracted operations reference genuine evidence present in the current user turn, normalize whitespace/case/punctuation, and reject/quarantine fabricated or stale evidence.

Output that was useful

- `normalize_text` for punctuation and whitespace collapse.
- `is_evidence_grounded` combining exact substring match and token overlap ratio.
- `filter_grounded_operations` to quarantine ungrounded operations.

What I questioned

- *What if the user says 'Jane Smith.' and the model extracts 'Jane Smith'?*
Answer: Punctuation-agnostic normalization ensures valid responses aren't rejected due to trailing periods, commas, or extra whitespace.
- *What if the model extracts evidence from a previous conversational turn?*
Answer: Grounding strictly validates against the *current* user turn, preventing old statements from re-triggering mutations without current intent.

Decision

Quarantine any operation whose evidence fails grounding against the current user message, leaving state untouched.

Result

- 6 unit tests in `tests/unit/test_grounding.py` covering exact matches, punctuation/whitespace normalization, token overlap, and rejection of fabricated/stale evidence.
- All 54 tests pass.

Log Entry 012 — F11 Robust LLM Response Parsing + Recovery

Prompt / Task

Implement F11: Single parsing utility in `app/llm/parsing.py` supporting Markdown code fence stripping, JSON extraction from surrounding text, Pydantic validation, and a single-retry repair loop with safe failure containment.

Output that was useful

- `extract_json_from_text` isolating clean JSON from Markdown fences and commentary.
- `extract_with_repair` giving the LLM one opportunity to correct malformed output with error details before safely aborting.

What I questioned

- *What happens if the second LLM response is also malformed?*

Answer: Return `(None, error)` cleanly so the conversation service leaves state unchanged and responds with a friendly retry message. Never allow a partial or corrupted state mutation.

Decision

Enforce a 2-strike parsing policy: attempt 1 $\rightarrow$ repair attempt 2 $\rightarrow$ friendly fallback if still failing.

Result

- 6 unit tests in `tests/unit/test_parsing_recovery.py` verifying plain JSON, code fences, text commentary, repair success, and safe two-failure handling.
- All 60 tests pass.

Log Entry 013 — F12 Conversation Service

Prompt / Task

Implement F12: Turn orchestration service in `app/services/conversation.py`. Wire together incoming messages, extraction + repair, evidence grounding, contradiction checks, deterministic reducer state updates, planner action selection, response composition, and safe fallback containment.

Output that was useful

- Clean orchestrator pipeline managing the complete turn boundary.
- Multiple fields in a single message captured atomically in one turn.
- Contradiction detection pausing mutations and preserving confirmed state.

What I questioned

- *If extraction fails completely due to invalid JSON or LLM provider errors, how should the conversation recover?*

Answer: Leave state untouched (0 changes), record the turn cleanly, and return a polite fallback asking the user to repeat or rephrase.

Decision

Keep the domain reducer and planner completely decoupled from I/O and LLM providers; `ConversationService` serves as the single orchestrator.

Result

- 5 integration tests in `tests/unit/test_conversation_service.py` verifying multi-field intake, contradiction pause, corrections, malformed response handling, and provider error safety.
- All 65 tests pass.

Log Entry 014 — F13 Natural-Language Response Composer

Prompt / Task

Implement F13: Dedicated `ResponseComposer` in `app/llm/composer.py`. Pass explicit `PlannerAction` instructions to the LLM for natural phrasing, while enforcing a deterministic fallback template matrix on any model errors, timeouts, or empty responses.

Output that was useful

- `ResponseComposer` decoupling response phrasing from domain state control.
- `DETERMINISTIC_ACTION_TEMPLATES` ensuring the assistant is never left speechless or stalled even when the LLM is unresponsive.

What I questioned

- *Can the composer decide to ask for a different field?*

Answer: No. The composer prompt and contract strictly bound the LLM to phrase the planner's selected action without altering workflow or state.

Decision

Give the LLM conversational flexibility to sound natural and polite, but surrender zero state control.

Result

- 4 unit tests in `tests/unit/test_response_composer.py` verifying template completeness and fallback resilience.
- All 69 tests pass.

Log Entry 015 — F14 Deterministic Document Renderer

Prompt / Task

Implement F14: Pure deterministic document renderer in `app/docgen/renderer.py`. Guarantee that generated documents are derived solely from confirmed structured state (no LLM, no reading chat history), always inject persistent legal disclaimers, format lists cleanly, and mark completion status accurately.

Output that was useful

- `render_wishes_document` outputting `RenderedDocument(title, text, is_complete, missing_fields)`.
- Mandatory disclaimers (`FICTIONAL PERSONAL WISHES DRAFT` , `NOT LEGAL ADVICE` , `DOES NOT CONSTITUTE LEGAL ADVICE`) baked into document text.

What I questioned

- *Should document rendering use the LLM to write eloquent legal prose?*

Answer: No. PRD explicitly requires deterministic generation from structured confirmed state to prevent

hallucinations, omissions, or embellishments.

Decision

Render document text directly from structured state using deterministic formatting and explicit disclaimers.

Result

- 4 unit tests in `tests/unit/test_docgen.py` verifying disclaimer injection, full completion, empty list representation ("None specified"), and contradiction containment.
- All 73 tests in the test suite pass.

Log Entry 016 — F15 Session Store + Per-Session Locking

Prompt / Task

Implement F15: In-memory `SessionStore` in `app/services/session_store.py` with per-session `asyncio.Lock` concurrency serialization. Ensure concurrent requests targeting the same session (e.g. rapid double-clicks) are executed sequentially without state corruption, while different sessions proceed in parallel without shared blocking.

Output that was useful

- `SessionStore` managing session lifecycle (create, get, save, delete, list).
- Per-session `asynccontextmanager` locking (`store.lock(session_id)`).

What I questioned

- *Should we use Redis or distributed locking?*
Answer: No. PRD explicitly marks Redis/distributed locks as a non-goal for this assessment. An in-memory dictionary with fine-grained `asyncio.Lock` per session ID is clean, fast, and fully sufficient.

Decision

Use in-memory `SessionStore` with per-session locking for deterministic concurrency control.

Result

- 3 concurrency and CRUD unit tests in `tests/unit/test_session_store.py` verifying serialization of concurrent requests to the same session and parallel execution across distinct sessions.
- All 76 tests in the test suite pass.

Log Entry 017 — F16 FastAPI Session Endpoints

Prompt / Task

Implement F16: REST API endpoints in `app/api/routes.py` and schemas in `app/api/schemas.py` (`POST /api/sessions`, `GET /api/sessions/{id}`, `POST /api/sessions/{id}/messages`, `GET`

`/api/sessions/{id}/document` , `DELETE /api/sessions/{id}`). Ensure message handling uses per-session concurrency locks and returns full turn states with rendered draft documents.

Output that was useful

- Clean separation between internal domain state models and external API transport schemas.
- Consistent endpoint contracts with proper HTTP status codes (200, 201, 404, 422).

What I questioned

- *Should API response schemas be distinct from domain models?*

Answer: Yes. Separating API transport schemas (`app/api/schemas.py`) from domain representations ensures changes to internal state models do not unintentionally break public API contracts.

Decision

Wrap message turn processing inside `session_store.lock(session_id)` and return synchronized state, changes, and document preview in every turn response.

Result

- 3 end-to-end HTTP integration tests in `tests/integration/test_api_endpoints.py` .
- All 79 tests pass.

Log Entry 018 — F17 Manual Correction API

Prompt / Task

Implement F17: Direct manual correction endpoint `PATCH /api/sessions/{session_id}/state` . Ensure manual edits from UI bypass the LLM completely, reuse the deterministic reducer & validators, record `kind="correction"` with `evidence="Manual UI Edit"` , clear pending clarifications for that field, and regenerate the draft document immediately.

Output that was useful

- `PATCH /api/sessions/{id}/state` providing a deterministic administrative/user override route.
- Fast execution path that guarantees manual inputs are directly applied without LLM latency or hallucination risks.

What I questioned

- *Should manual edits be routed through an LLM to normalize text?*

Answer: No. Direct manual corrections from the UI represent explicit user intent and must go directly through the deterministic reducer and validators.

Decision

Direct UI edits execute as pure state reducer operations with locked concurrency and immediate document regeneration.

Result

- 3 integration tests in `tests/integration/test_manual_correction.py` verifying direct edits, rejection of invalid fields/types (422), and immediate document synchronization.
 - All 82 tests in the test suite pass.
-

Log Entry 019 — Multi-Field Extraction Fix & Regression Test (F12)

Prompt / Task

Address bug discovered during manual testing: When the user sends a multi-field message like *"I have assets around the world and I have two children named Aarav and Anaya."*, the extractor was falling back to storing the entire sentence in `additional_wishes` instead of emitting independent operations for `covers_worldwide_assets`, `has_children`, and `children`.

Output that was useful

- Broadened regex and pattern extraction for `covers_worldwide_assets` to match `"around the world"`, `"across the world"`, `"global"`, etc.
- Extended `has_children` and `children` extraction to support spelled-out numbers (e.g. `"two children"`) and name lists.
- Strictly gated `additional_wishes` extraction to actual wish/funeral/cremation/burial keywords so that unstructured text does not pollute the wishes list.

What I questioned

- *Why did the extractor treat unrecognized multi-field statements as additional wishes?*
Answer: `mock.py` contained an overly permissive fallback that treated any long message with 0 initial matches as an additional wish. Replacing this with strict keyword and intent gating eliminates the issue.

Decision

Ensure each recognized domain pattern produces an independent grounded operation atomically, and never use `additional_wishes` as a generic fallback.

Result

- Added regression tests in `tests/integration/test_api_endpoints.py` and `tests/unit/test_conversation_service.py`.
 - Verified that *"I have assets around the world and I have two children named Aarav and Anaya."* correctly produces `covers_worldwide_assets: true`, `has_children: true`, `children: ["Aarav", "Anaya"]`, and leaves `additional_wishes` untouched.
 - All 84 tests pass.
-

Log Entry 020 — Bidirectional Children Contradiction Fix & Regression (F05)

Prompt / Task

Address F05 regression found during manual testing: When state has `has_children=True` confirmed (with children), sending an unacknowledged negation like *"Actually, I don't have any children."* must produce an operation with `is_correction=False` and trigger a `PendingClarification(issue_type="contradiction")` while keeping the confirmed state intact. Also verify the reverse direction (`has_children=False` $\rightarrow$ *"I actually have a son named Aarav."*).

Output that was useful

- Enhanced `MockLLMClient` to output uncorrected operations for explicit negation statements so they route to the domain contradiction engine.
- Strengthened `detect_contradictions` in `app/domain/contradictions.py` with bidirectional checks:
- `has_children=True` $\rightarrow$ `has_children=False` statements.
- `has_children=False` $\rightarrow$ mentioning children, sons, or daughters.
- Natural question formatting for boolean/family contradictions.

What I questioned

- Should the system silently overwrite confirmed `has_children=True` and delete children when the user says 'I don't have children'?*
Answer: No. PRD F05 guardrail explicitly states: *Contradiction $\neq$ Correction. An unacknowledged conflict must pause and create a `PendingClarification`. State must remain confirmed until the user explicitly resolves the ambiguity.*

Decision

Enforce bidirectional contradiction detection across boolean family fields and test both directions.

Result

- Added unit test in `tests/unit/test_contradictions.py` and two API regression tests in `tests/integration/test_api_endpoints.py`.
 - All 87 tests pass.
-

Log Entry 021 — Three-Column Functional Frontend UI (F18–F22)

Prompt / Task

Implement Feature F18 (Three-Column Chat UI):

- Column 1: Intake Dialogue (Chat Stream, dynamic message bubbles, input form, contradiction alert banner).
- Column 2: Structured State Inspector (status badges for CONFIRMED, UNCONFIRMED, UNKNOWN, NOT_APPLICABLE, audit change log, raw JSON view, direct manual state editor).

3. Column 3: Live Document Preview (real-time generated markdown, mandatory legal disclaimer notice, completion ratio badge, copy button).
4. Serve static frontend assets (`static/index.html` , `static/styles.css` , `static/app.js`) from FastAPI and mount static directory.

Output that was useful

- Clean modern dark UI theme with typography from Google Fonts (`Plus Jakarta Sans` , `JetBrains Mono`).
- Responsive 3-column layout built with Vanilla CSS and vanilla JS (no heavy frontend framework).
- Integrated direct manual override form connected to `PATCH /api/sessions/{session_id}/state` for instantaneous state corrections.

What I questioned

- *Should static files override or conflict with API endpoints?*

Answer: No. The `api_router` is included prior to static mounts, ensuring all `/api/*` endpoints resolve reliably.

Decision

Mount static directory at `/static` and root `/` with `html=True` using `fastapi.staticfiles.StaticFiles` .

Result

- Created `static/index.html` , `static/styles.css` , `static/app.js` .
- Updated `app/main.py` to mount static directory.
- Created `tests/integration/test_static_frontend.py` and updated `tests/test_health.py` .
- All 90 tests pass.

Log Entry 022 — Golden Scenario Suite (F23)

Prompt / Task

Implement Feature F23 (Golden Scenario Test Suite):

1. Scenario 1: *Sequential Happy Path* through all fields to full document completion.
2. Scenario 2: *Atomic Multi-field Turn* with worldwide assets and named children.
3. Scenario 3: *Contradiction Detection, Pause, & Explicit Resolution*.
4. Scenario 4: *Mid-session Explicit Correction* with audit logging.
5. Scenario 5: *Unrelated Chit-chat / Hallucinated Input Rejection*.
6. Scenario 6: *Conditional Cascade Rules* (`has_children=False` $\rightarrow$ `children=NOT_APPLICABLE` , subsequent correction resets to `UNKNOWN`).
7. Scenario 7: *Direct Manual State Override* (`PATCH /api/sessions/{id}/state`).
8. Scenario 8: *LLM Failure Recovery & Graceful Fallback*.

Output that was useful

- All 8 comprehensive golden scenarios implemented in `tests/scenarios/test_golden_scenarios.py` .

- Refined extraction layer to distinguish explicit correction keywords (`correction` , `mistake` , `change my`) from soft conversational words (`actually`) when evaluating sensitive boolean flips.
- Ensured cross-field contradiction checks in `app/domain/contradictions.py` respect explicit correction flags (`is_correction=True`).

What I questioned

- *Should soft conversational words like 'actually' allow bypassing contradiction checks on cascade-triggering fields?*

Answer: No. Negation of children/family status requires explicit correction or clear resolution to safeguard downstream state integrity.

Decision

Enforce strict separation between unacknowledged conflicting statements and explicit corrections across both domain engine and mock/extraction layers.

Result

- Created `tests/scenarios/test_golden_scenarios.py` with 8 passing end-to-end scenario tests.
- Full test suite has 98 passing tests (`pytest -v`).

Log Entry 023 — Architectural Decisions & Blueprint (F25)

Prompt / Task

Author `DECISIONS.md` capturing all architectural choices, trade-offs, and future production roadmap:

1. Deterministic Domain Engine with LLM-as-Extractor vs autonomous agent frameworks (LangGraph/CrewAI).
2. Domain state `wishesState` as the single source of truth vs conversation history.
3. Evidence grounding and 1-attempt repair flow.
4. Contradiction detection vs explicit correction semantics.
5. Conditional cascades and rules engine.
6. Deterministic document rendering with mandatory legal notices.
7. Concurrency isolation with per-session async locks.
8. Production scaling roadmap (PostgreSQL event-sourcing, Redis distributed locks, OAuth2, OpenTelemetry).

Output that was useful

- Comprehensive, structured `DECISIONS.md` created at workspace root.

What I questioned

- *Why not let the LLM generate the final legal document directly?*

Answer: Direct LLM generation risks omitting legally mandated disclaimers or hallucinating clauses. A deterministic templating engine ensures 100% legal compliance and transparency.

Decision

Document the deterministic architecture and production blueprint in `DECISIONS.md`.

Result

- Created `DECISIONS.md`.

Log Entry 024 — Comprehensive Documentation & Quickstart (F26)

Prompt / Task

Author `README.md` providing:

1. System overview and design philosophy.
2. Architecture flowchart and component breakdown.
3. Quickstart instructions (virtualenv, dependencies, configuration, test runner, dev server).
4. Mode selection guide (offline `MockLLMClient` vs live `GeminiClient` with `google-genai`).
5. Complete API endpoint summary and Three-Column UI walkthrough.
6. Repository structure.

Output that was useful

- Clean, structured markdown documentation covering setup, testing, and production deployment.

What I questioned

- *Should README assume an active Gemini key is mandatory for evaluation?*

Answer: No. The system runs out-of-the-box in `MockLLMClient` mode with 100% test pass rate offline, while supporting 1-variable activation (`LLM_PROVIDER=gemini`) for live Gemini integration.

Decision

Document both mock and live Gemini workflows clearly in `README.md`.

Result

- Created `README.md`.
- All 98 tests pass across unit, integration, and golden scenario suites.

Log Entry 025 — Manual Session Fixes: Address Fallback, Role vs Name Separation, & Contradiction Resolution

Prompt / Task

Fix 3 manual testing regressions:

1. **Repeated Address Loop:** Unformatted / institutional locations (e.g. "International Institute of Information Technology, Pune", "allbaskapur") were rejected by strict regexes, trapping the user in an address loop.
2. **Role Descriptor Stored as Executor Name:** Descriptive relationship roles (e.g. "my mistress", "my

lover", "my future wife") were parsed as `executor.name`.

3. Stuck Contradiction Loop: When clarifying a contradiction, the new clarifying statement was treated as a secondary contradiction against old confirmed state, trapping the session.

Output that was useful

- Relaxed address fallback in `MockLLMClient` to accept non-empty text when `home_address` is unknown and `full_name` is known.
- Separated relationship descriptors (`mistress`, `lover`, `wife`, `spouse`, etc.) to `executor.relationship`, leaving `executor.name` unknown until a proper name is provided.
- Fixed `ConversationService`, `detect_contradictions`, and `reducer.py` so that answering a pending clarification resolves the contradiction, applies the new state with `is_correction=True`, and clears `pending_clarification`.

What I questioned

- *Should providing a real name after an informal relationship descriptor trigger a contradiction?*
Answer: No. If `executor.name` was unknown and the user was only describing the role (e.g., "my mistress"), providing the actual name and relationship ("Her name is Emily, she is my spouse") completes the executor's identity.

Decision

Treat active `pending_clarification` responses as intent resolutions and allow smooth state advancement.

Result

- Created `tests/integration/test_manual_session_regressions.py` covering all 3 scenarios.
- All 101 tests pass across unit, integration, and scenario suites.

Log Entry 026 — Google Gemini API Configuration & Readiness

Prompt / Task

Prepare files and environment for live Google Gemini API integration: create `.env` with `LLM_PROVIDER=gemini` and `GEMINI_API_KEY=` placeholder, refine `GeminiClient` with temperature controls (0.0 for deterministic JSON extraction, 0.7 for conversational composition) and robust error handling using the official `google-genai` SDK.

Output that was useful

- Pre-configured `.env` file with `LLM_PROVIDER=gemini`, ready for immediate key insertion.
- Updated `GeminiClient` in `app/llm/gemini.py` wrapping `google.genai.Client` with `types.GenerateContentConfig` for structured schema enforcement and temperature control.
- Clear error handling for 429 rate limits, invalid API keys, and timeouts.

What I questioned

- *Should we leave LLM_PROVIDER as mock if no key is entered?*

Answer: Yes, the dependency provider in `app/api/routes.py` gracefully falls back to `MockLLMClient` if `GEMINI_API_KEY` is empty, avoiding runtime startup crashes while making switching instantaneous once the key is pasted.

Result

- `.env` file created and ready for user API key.
 - `app/llm/gemini.py` verified with `google.genai` SDK.
 - 101/101 test suite passing.
-

Log Entry 027 — Robust Markdown Fence Stripping & Free-Tier Model Configuration

Prompt / Task

Fix address input fallback issue where Gemini wrapped JSON inside `json ...` fences, causing raw text parser rejection and 0 operations extracted. Also configure active available free-tier model (`gemini-3.5-flash-lite`) to prevent daily rate-limit exhaustion.

Output that was useful

- Upgraded `extract_json_from_text` in `app/llm/parsing.py` to use `re.DOTALL` code fence extraction, outermost brace extraction, and trailing comma repair.
- Updated `GEMINI_MODEL=gemini-3.5-flash-lite` with active RPM/RPD quota.
- Added test environment isolation in `tests/conftest.py` ensuring fast offline mock testing.

Result

- Successfully tested and verified live extractions on the reported address examples ("International Institute of Information Technology , Pune", "I live at I2IT college , which is situated at Hinjewadi in Pune", and "123 Main Street, Apt 4B, New York, NY 10001, United States").
 - All 101 unit and integration tests passing.
-

Log Entry 028 — Wrap-up & Negative Responses for Additional Wishes & Finalization

Prompt / Task

Fix conversational loop at the end of the intake where answers like "nothing", "not much", "nothing please finalize", or "no baba" were classified by Gemini as "unclear" ambiguities, leaving `additional_wishes` as `UNKNOWN` and indefinitely repeating the final wishes prompt.

Output that was useful

- Enhanced `SYSTEM_EXTRACTION_PROMPT` in `app/llm/prompts.py` to explicitly map concise negative and closing words ("nothing", "not much", "none", "no", "no baba", "finalize", "all done") to `additional_wishes = []` (confirmed empty list).
- Supported free-form wish statements (e.g. "sex is very nice", "play jazz at my funeral") into `additional_wishes`.
- Updated `MockLLMClient` with matching closing intent patterns.

Result

- Added regression test `test_regression_additional_wishes_concise_negative_answers_complete_session` in `tests/integration/test_manual_session_regressions.py`.
 - 102/102 test suite passing.
-

Log Entry 029 — History Context & Contextual Affirmation/Denial Handling

Prompt / Task

Fix conversational loop where single-word affirmative answers (e.g., "yes", "yes I do own", "yeah") to questions about worldwide assets or children were classified as generic confirmation without operations because the extraction prompt lacked the recent question history context.

Output that was useful

- Injected `RECENT CONVERSATION HISTORY` (recent turns) directly into `build_extraction_prompt` in `app/llm/prompts.py`.
- Added explicit contextual rules for mapping affirmative/negative answers ("yes", "no", "yeah", "nope", "I do", "only domestic") to `covers_worldwide_assets`, `has_children`, `specific_gifts`, and `additional_wishes` based on the active question.
- Made `ExtractionAmbiguity` and `ExtractionContradiction` in `app/llm/schemas.py` resilient with defaults to prevent Pydantic validation schema failures.

Result

- Live Gemini tests verified: "yes", "yes I do own", "no", "yeah", "nope" accurately extract to `covers_worldwide_assets` with high confidence.
 - 102/102 tests passing.
-

Log Entry 030 — Structured Completion Summary & Elimination of Post-Completion Loops

Prompt / Task

Fix post-intake conversational loop where, once the document was complete, the assistant asked "Would you like me to generate a summary?", and when the user answered "yes" or "give me complete summary", the assistant repeatedly re-asked the same question instead of outputting the summary.

Output that was useful

- Passed full `state_summary` in `action_context` when `plan_next_action` returns `PlannerAction.COMPLETE`.
- Injected `recent_messages` into `build_compose_prompt` in `app/llm/prompts.py`.
- Instructed response composer that upon `COMPLETE`, if user asks for a summary or agrees, it must generate a clean bulleted breakdown of all recorded wishes and direct the user to the Document Preview panel.

Result

- Verified live with Gemini: when user requests a summary, assistant outputs full structured bulleted summary without looping.
 - 102/102 test suite passing.
-

Log Entry 031 — Final System Verification & End-to-End Golden Flow Validation

Prompt / Task

Conduct final end-to-end conversational intake session across all fields, edge cases, contradiction resolutions, unformatted institutional addresses, multi-turn role separation, closing negative phrases, and summary generation. Verify system stability, 102/102 test suite integrity, and synthesize production improvement recommendations.

Output that was useful

- Validated the complete end-to-end intake flow in live environment:
 1. Full Name captured & formatted (`Gandu Sharma`).
 2. Unformatted address accepted without loop (`i2it , hinjewadi , Pune`).
 3. Contextual affirmative answered worldwide coverage on turn 1 (`yes`).
 4. Contextual negative answered children & auto-cascaded (`no`).
 5. Role vs Name separation correctly recorded (`my wife -> Sunny Leone`).
 6. Closing negative phrases confirmed cleanly (`nope , nothing as such`).
 7. Requested summary generated immediately as structured bullets.
 8. Natural conversational signoff.

What I questioned

- *What architectural enhancements are needed before taking this to production?*

Answer: Detailed below in the Production Improvements section (persistent storage, streaming responses, PII protection, document exports, and LLM evaluation).

Result

- 102/102 unit, integration, and scenario tests passing.
- Clean git repository state.

Production Improvements & Roadmap

If transitioning this intake assistant into a production environment, the following practical enhancements are recommended:

1. Persistent Session Storage

- **Current State:** In-memory `SessionStore` with per-session `asyncio.Lock()`.
- **Production Enhancement:** Transition to **PostgreSQL** (storing structured state and change history in JSONB columns) with SQLAlchemy / AsyncPG, paired with **Redis** for distributed session locking and caching.

2. Streaming Conversational Responses (SSE)

- **Current State:** Standard request-response cycle waiting for the full LLM response.
- **Production Enhancement:** Implement **Server-Sent Events (SSE)** to stream conversational assistant messages token-by-token for lower perceived latency, while structured JSON extraction continues to run deterministically.

3. PII Protection & Data Privacy

- **Current State:** Cleartext state models.
- **Production Enhancement:** Apply field-level encryption for sensitive PII (names, addresses, beneficiary information) at rest in the database, along with configurable data retention and session expiration policies.

4. Document Export (PDF / Formatted Downloads)

- **Current State:** Plain text / markdown document preview in the UI with copy functionality.
- **Production Enhancement:** Add clean PDF and DOCX export capabilities using lightweight rendering libraries (e.g. WeasyPrint or Jinja2 templates) so users can download and print their draft wishes directly.

5. Continuous LLM Evaluation & Grounding Monitoring

- **Current State:** 102 deterministic offline unit, integration, and golden scenario tests.
- **Production Enhancement:** Establish an automated offline evaluation pipeline to benchmark extraction accuracy across prompt updates, tracking groundedness metrics and catching prompt regressions automatically.